有道翻译 支持免费文档翻译60万字符/年 立即下载→ https://fanyi.youdao.com/download

arXiv:2310.16985v4 [cs.RO] 12 Aug 2025

TinyMPC：面向资源受限微控制器的模型预测控制

Anoushka Alavilli*, Khai Nguyen*, Sam Schoedel*, Brian Plancher, Zachary Manchester

摘要—模型预测控制（MPC）是控制受复杂约束的高动态机器人系统的一种强大工具。然而，MPC计算开销较大，通常难以在资源受限的小型机器人平台上实现。本文提出TinyMPC，这是一种专为小型机器人常用微控制器设计的高速MPC求解器，具有极低的内存占用。我们的方法基于交替方向乘子法（ADMM），并利用MPC问题的结构以提高效率。我们通过与当前最先进的求解器OSQP进行基准测试，验证了TinyMPC的有效性，其速度提升了近一个数量级；并通过在27克四旋翼飞行器上的硬件实验，展示了其在高速轨迹跟踪和动态避障方面的性能。TinyMPC已公开发布于**https://tinympc.org**。

I. 引言

模型预测控制（MPC）能够在满足复杂控制和状态约束（例如动态避障和接触事件中遇到的约束）的同时，实现机器人响应迅速且动态的在线控制[1]，[2]，[3]，[4]。然而，尽管MPC取得了诸多成功，其实际应用常常受到计算能力限制的阻碍，这可能需要对算法进行简化[5]，[6]。当处理具有快速或不稳定的开环动力学的系统时，这一挑战尤为突出，因为安全高效运行需要较高的控制频率。

与此同时，人们对能够在狭小空间内运行的微型、低成本机器人产生了巨大兴趣，使其成为从应急搜救[7]到基础设施和设备常规监测与维护[8][9]等应用中的有前景的解决方案。这些机器人在计算方面受限于低功耗、资源受限的微控制器（MCU）[10][11]。如图2所示，与大型机器人所使用的CPU和GPU相比，这些微控制器的RAM、闪存容量和处理器速度相差几个数量级，历史上一直无法支持计算密集型或内存密集型算法的实时执行[12][13]。因此，文献中许多关于智能

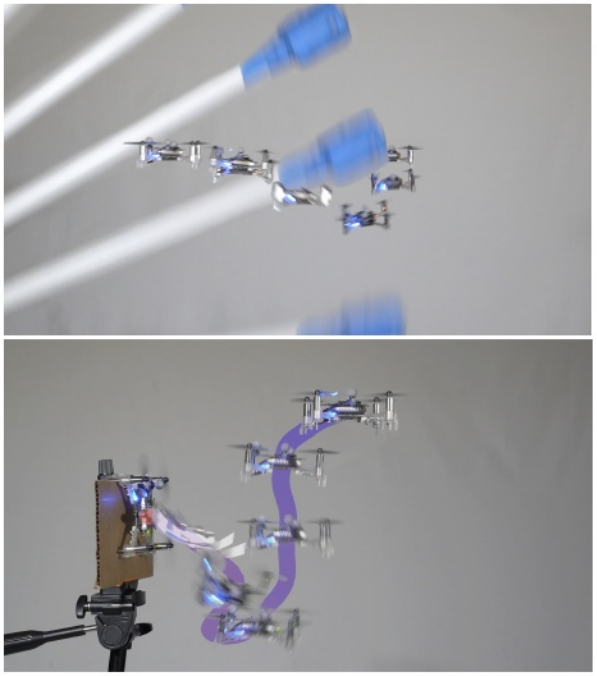


图1. TinyMPC是一种快速的凸模型预测控制求解器，可在资源受限的微控制器上实现实时最优控制。我们在27克重的Crazyflie四旋翼飞行器上展示了其在动态避障（上图）和从90°姿态误差中恢复（下图）方面的有效性。

这些微型平台上执行的智能机器人行为依赖于外部计算机[14]，[15]，[16]，[17]，[18]，[19]，[20]。

近年来，已涌现出多种适用于嵌入式模型预测控制（MPC）的高效优化求解器和技术[29]、[30]。其中较为著名的软件包包括OSQP[31]和CVXGEN[32]。这两类求解器均具备代码生成工具，可帮助用户生成无需依赖外部库的C语言代码，在嵌入式计算机上求解二次规划（QP）问题。然而，它们未能充分利用MPC问题特有的结构，通常导致内存占用过高、计算开销过大，难以在许多微控制器的资源限制下运行。

另一方面，近年来“TinyML”的成功使得神经网络得以在微控制器上部署[12]。受这些成果的启发，我们开发了TinyMPC，这是一种针对微控制器优化的凸型模型预测控制（MPC）实现，采用交替方向乘子法（ADMM）算法。我们的求解器核心设计在于利用MPC问题的结构特性，对ADMM算法进行加速和压缩。

特别是，我们预先计算并缓存了代价高昂的矩阵分解，使TinyMPC能够完全避免在线计算。

*These authors contributed equally and are listed in alphabetical order.

Anoushka Alavilli is with the Department of Electrical & Computer Engineering, Carnegie Mellon University, 5000 Forbes Ave., Pittsburgh, PA 15213. apalavil@andrew.cmu.edu

Khai Nguyen is with the Department of Mechanical Engineering, Carnegie Mellon University, 5000 Forbes Ave., Pittsburgh, PA 15213. xuankhan@andrew.cmu.edu

Samuel Schoedel and Zachary Manchester are with the Robotics Institute, Carnegie Mellon University, 5000 Forbes Ave., Pittsburgh, PA 15213. {sschoede, zmanches}@andrew.cmu.edu

Brian Plancher is with Barnard College, Columbia University, 3009 Broadway New York, NY 10027. bplancher@barnard.edu

	Micro Platforms		Tiny Platforms				Full-Scale Platforms	
	Robobee	HAMR-F	Crazyflie2.1	DeepPicar Micro	PIXHAWK PX4	Petoi Bittle	Snapdragon Flight	Unitree Go1edu
Processor	ATtiny20 4-8 MHz 8-bit MCU	ATmega1284RF2 16MHz 8-bit MUC	STM32F405 168 MHz 32-bit M4 MCU	RP2040 133 MHz Dual-Core 32-bit M0+ MCU	STM32F765 216 MHz Dual-Core 32-bit M7 MCU	ESP32-WROOM-32D 240MHz Dual-Core 32-bit LX7 MCU	Qualcomm Snapdragon 801 2.15 GHz Quad-Core 32-bit CPU 450 MHz 32-pipeline GPU	Jetson Nano (x3) 1.43 GHz Quad-Core 64-bit CPU 921 MHz 128-core GPU
RAM	128 B	16 kB	196 kB	264 kB	512 kB	512 kB	2 GB	4 GB (x3)
Flash	2 kB	128 kB	1 MB	2 MB	2 MB	16 MB	32 GB	64-256 GB (via SD card x3)
Processor Power	0.015 W	0.045 W (with RF)	0.15 W	0.15 W	0.5 W	0.5-1 W	3-10 W	5-10 W (x3)

图2. 微型、小型和全尺寸机器人平台及其相关计算硬件的对比。在最小尺度上，如Robobee[21]和HAMR-F[22]等微型机器人使用高度受限的8位微控制器来执行预先规划的开环步态或翅膀运动。在较大尺度上，Snapdragon Flight四旋翼[23]或Unitree Go1edu四足机器人[24]所搭载的强大嵌入式CPU和GPU可实现高性能，但代价是较高的功耗需求。本研究针对的是Crazyflie2.1[25]、DeepPiCarMicro[26]、PIXHAWK PX4[27]和Petoi Bittle[28]等小型机器人，它们采用32位微控制器进行运动规划与控制。这些设备具备一定的机载计算能力，但其处理器速度、RAM和闪存容量相比全尺寸机器人低了几个数量级。

除法和矩阵求逆。该方法能够在极小的内存占用下实现快速计算，从而支持在资源受限的微控制器（MCU）上部署。据作者所知，TinyMPC 是首个专为在这些 MCU 上运行而设计，并已在高度动态且计算资源有限的机器人系统中成功验证的模型预测控制（MPC）求解器。我们的贡献包括：

- 一种专为模型预测控制（MPC）优化的新型二次规划算法，无需矩阵求逆，具有高效率和极低的内存占用。这一组合使其非常适合在资源受限的微控制器上部署。
- 一个用 C++ 编写的 TinyMPC 开源实现，可在微控制器上为凸型模型预测控制问题提供最先进的实时性能。
- 在小型、灵活且资源受限的四旋翼平台上进行实验演示。

本文结构如下：第二部分回顾了线性二次最优控制、凸优化和ADMM。第三部分推导了核心的TinyMPC求解器算法。第四部分展示了在Crazyflie四旋翼无人机上的基准测试结果和硬件实验。最后，第五部分总结了我们的研究结果与结论。

II. 背景

线性二次调节器

线性二次调节器（LQR）[33] 是一种广泛用于解决机器人控制问题的方法。LQR 在一组线性动力学约束条件下，对二次代价函数进行优化：

$$\begin{aligned}\min_{x_{1:N},u_{1:N-1}} \; & J = \frac{1}{2}x_N^\intercal Q_f x_N + q_f^\intercal x_N + \\ & \sum_{k=1}^{N-1}\frac{1}{2}x_k^\intercal Q x_k + q_k^\intercal x_k + \frac{1}{2}u_k^\intercal R u_k + r_k^\intercal u_k \\ \text{subject to} \quad & x_{k+1} = Ax_k + Bu_k \quad \forall k \in [1, N),\end{aligned} \tag{1}$$

其中 $x_k \in R^n$，$u_k \in R^m$ 分别表示第 k 个时间步的状态和控制输入，N 为时间步数（也称为预测时域），A ∈ Rn×n 和 B ∈ Rn×m 定义了系统的动态特性，Q ⊠ 0、R ⊠ 0 和 Q_f ⊠ 0 是对称的成本矩阵，q 和 r 是线性成本向量。

方程（1）具有以仿射反馈控制器形式表示的闭式解 [33]：

$$u_k^* = -K_k x_k - d_k. \tag{2}$$

反馈增益 K_k 和前馈增益 d_k 通过从 PN = Qf、pN = qf 开始，沿时间反向求解离散Riccati方程得到，其中 Pk 和 pk 分别是代价函数（或价值函数）的二次项和一次项 [33]：

$$\begin{aligned}K_k &= (R + B^\intercal P_{k+1} B)^{-1}(B^\intercal P_{k+1} A) \\ d_k &= (R + B^\intercal P_{k+1} B)^{-1}(B^\intercal p_{k+1} + r_k) \\ P_k &= Q + K_k^\intercal R K_k + (A - BK_k)^\intercal P_{k+1}(A - BK_k) \\ p_k &= q_k + (A - BK_k)^\intercal (p_{k+1} - P_{k+1} B d_k) + \\ &\quad K_k^\intercal (R d_k - r_k).\end{aligned} \tag{3}$$

B. 凸模型预测控制

凸型MPC将LQR的公式扩展，以允许对系统状态和控制输入施加额外的凸约束，例如关节和力矩限制、用于避障的超平面以及接触约束：

$$\begin{aligned}\min_{x_{1:N},u_{1:N-1}} \; & J(x_{1:N}, u_{1:N-1}) \\ \text{subject to} \quad & x_{k+1} = Ax_k + Bu_k, \\ & x_k \in \mathcal{X}, u_k \in \mathcal{U},\end{aligned} \tag{4}$$

其中 X 和 U 是凸集。该问题的凸性意味着它可以被高效且可靠地求解，从而能够在多种控制应用中实现实时部署，包括火箭着陆[34]、足式移动[35]和自动驾驶[36]。

当X和U可以表示为线性约束时，(4) 就是一个二次规划问题，可转化为标准形式：

$$\begin{aligned}\min_{x \in \mathbb{R}^n} \; & \frac{1}{2}x^\intercal P x + q^\intercal x \\ \text{subject to} \quad & Ax \le b, \\ & Cx = d.\end{aligned} \tag{5}$$

有关可行性与稳定性的理论保证及其他进一步分析，可参见[37]和[38]。

交替方向乘子法

交替方向乘子法（ADMM）[39]、[40]、[41] 是求解凸优化问题（包括如公式(5)所示的二次规划问题）的一种流行且高效的方法。此处我们仅作简要概述，更多细节请参见[42]。

给定一个通用问题：

$$\begin{aligned} \min_{x} \quad & f(x) \\ \text{subject to} \quad & x \in \mathcal{C}, \end{aligned} \tag{6}$$

当 f 和 C 为凸函数时，我们定义集合 C 的指示函数：

$$I_{\mathcal{C}}(z) = \begin{cases} 0 & z \in \mathcal{C} \\ \infty & \text{otherwise.} \end{cases} \tag{7}$$

现在我们可以通过引入松弛变量 z 来形成以下等价问题：

$$\begin{aligned} \min_{x} \quad & f(x) + I_{\mathcal{C}}(z) \\ \text{subject to} \quad & x = z. \end{aligned} \tag{8}$$

变换后问题（8）的增广拉格朗日函数如下，其中λ为拉格朗日乘子，ρ为标量惩罚权重：

$$\mathcal{L}_A(x, z, \lambda) = f(x) + I_{\mathcal{C}}(z) + \lambda^\intercal (x - z) + \frac{\rho}{2} ||x - z||_2^2. \tag{9}$$

如果我们交替对 x 和 z 进行最小化，而不是同时对两者进行最小化，就能得到三步 ADMM 迭代。

$$\text{primal update} : x^+ = \arg\min_{x} \mathcal{L}_A(x, z, \lambda), \tag{10}$$

$$\text{slack update} : z^+ = \arg\min_{z} \mathcal{L}_A(x^+, z, \lambda), \tag{11}$$

$$\text{dual update} : \lambda^+ = \lambda + \rho(x^+ - z^+), \tag{12}$$

最后一步是对拉格朗日乘子进行梯度上升更新[41]。这些步骤可以迭代，直到达到所需的收敛容差为止。

在二次规划（QP）的特殊情况下，ADMM算法的每一步计算都变得非常简单：原变量更新是求解一个线性系统，而松弛变量更新则是一个线性投影。基于ADMM的QP求解器，如OSQP[31]，已展现出最先进的性能。

III. THE TINYMPC 解决器

TinyMPC 通过利用模型预测控制（MPC）问题的特殊结构，以牺牲通用性来换取速度。具体而言，我们利用线性二次调节器（LQR）问题的解析型Riccati解来计算公式(10)中的原始变量更新。通过预先计算并缓存该解，我们可以避免在线进行矩阵分解，在保持较小内存占用的同时实现极快的运行性能。

A. 结合LQR与ADMM用于MPC

我们通过引入松弛变量（如公式(9)所示），将问题(4)转化为如下形式来求解：

$$\begin{aligned} \min_{\substack{x_{1:N}, z_{1:N}, \\ \lambda_{1:N}, u_{1:N-1}, \\ w_{1:N-1}, \mu_{1:N-1}}} \quad & \mathcal{L}_A(\cdot) = J(x_{1:N}, u_{1:N-1}) + \\ & I_{\mathcal{X}}(z_{1:N}) + I_{\mathcal{U}}(w_{1:N-1}) + \\ & \sum_{k=1}^{N} \frac{\rho}{2}(x_k - z_k)^\intercal (x_k - z_k) + \lambda_k^\intercal (x_k - z_k) + \\ & \sum_{k=1}^{N-1} \frac{\rho}{2}(u_k - w_k)^\intercal (u_k - w_k) + \mu_k^\intercal (u_k - w_k) \\ \text{subject to:} \quad & x_{k+1} = Ax_k + Bu_k, \end{aligned} \tag{13}$$

其中 z、w、λ 和 μ 分别是整个时域内的状态松弛变量、输入松弛变量、状态对偶变量和输入对偶变量。通过指示函数 I_X 和 I_U 来强制执行状态和输入约束。我们采用 ADMM 算法（10）、（11）、（12）来求解该最优控制问题。（13）式的原变量更新转化为一个等式约束的二次规划问题：

$$\begin{aligned} \min_{x_{1:N}, u_{1:N-1}} \quad & \frac{1}{2} x_N^\intercal \tilde{Q}_f x_N + \tilde{q}_f^\intercal x_N + \\ & \sum_{k=1}^{N-1} \frac{1}{2} x_k^\intercal \tilde{Q} x_k + \tilde{q}_k^\intercal x_k + \frac{1}{2} u_k^\intercal \tilde{R} u_k + \tilde{r}^\intercal u_k \end{aligned} \tag{14}$$

受控于 xk+1 = Axk + Buk,

在哪里

$$\begin{aligned} \tilde{Q}_f &= Q_f + \rho I, \qquad & \tilde{q}_f &= q_f + \lambda_N - \rho z_N, \\ \tilde{Q} &= Q + \rho I, & \tilde{q}_k &= q_k + \lambda_k - \rho z_k, \\ \tilde{R} &= R + \rho I, & \tilde{r}_k &= r_k + \mu_k - \rho w_k. \end{aligned} \tag{15}$$

我们对(15)式进行重写，并为方便起见引入缩放后的对偶变量y和g[41]:

$$\begin{aligned} \tilde{q}_f &= q_f + \rho(\lambda_N/\rho - z_N) = q_f + \rho(y_N - z_N), \\ \tilde{q}_k &= q_k + \rho(\lambda_k/\rho - z_k) = q_k + \rho(y_k - z_k), \\ \tilde{r}_k &= r_k + \rho(\mu_k/\rho - w_k) = r_k + \rho(g_k - w_k). \end{aligned} \tag{16}$$

我们注意到，由于(14)与(1)具有相同的LQR问题结构，因此可以中的Riccati递推高效求解。对于(13)的松弛变量更新，则变为对可行集的一个简单线性投影：

$$\begin{aligned} z_k^+ &= \text{proj}_{\mathcal{X}}(x_k^+ + y_k), \\ w_k^+ &= \text{proj}_{\mathcal{U}}(u_k^+ + g_k), \end{aligned} \tag{17}$$

其中上标表示后续ADMM迭代中的变量。对于(13)的对偶更新变为：

$$\begin{aligned} y_k^+ &= y_k + x_k^+ - z_k^+, \\ g_k^+ &= g_k + u_k^+ - w_k^+. \end{aligned} \tag{18}$$

当原问题和对偶问题的残差都在设定的容差范围内时，算法终止。

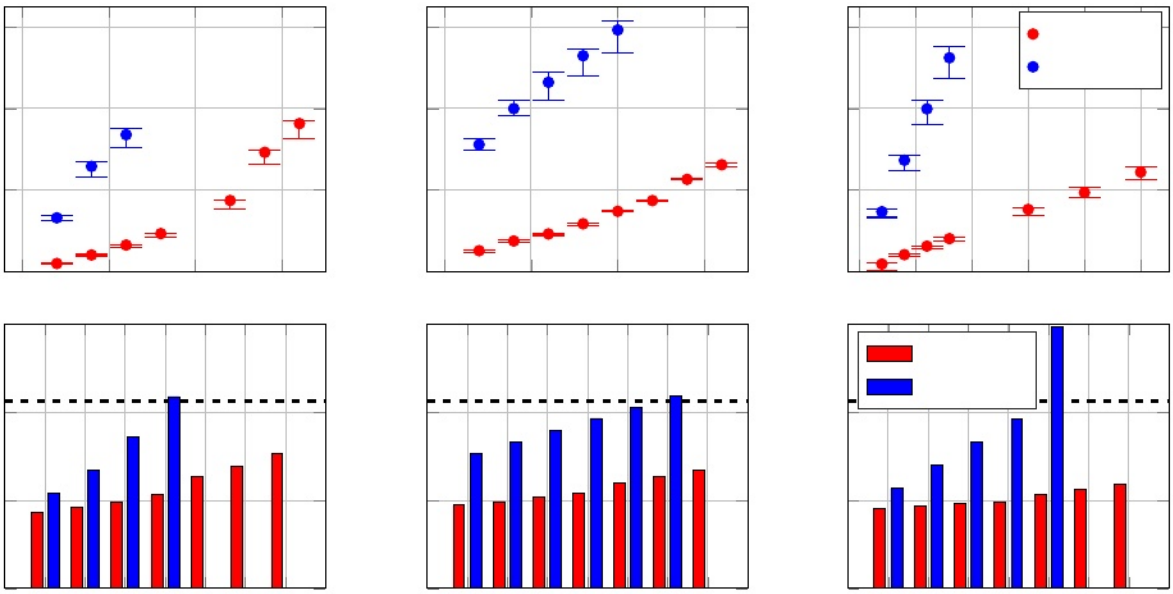


图3. 在Teensy 4.1开发板（ARM Cortex-M7，主频600 MHz，支持32位浮点运算，配备7.75 MB闪存和512 kB紧密耦合RAM）上，对随机生成的轨迹跟踪问题，比较OSQP与TinyMPC的平均迭代次数（上图）和内存使用情况（下图）。误差线表示针对给定问题执行的所有MPC步骤中每次迭代的最大和最小时间。在(a)中，输入维度m和时间步长N保持恒定，分别为m = 4、N = 10，而状态维度n从4变化到32；在(b)中，n = 10、N = 10，而m从4变化到32；在(c)中，n = 10、m = 4，N从4变化到50。虚线黑线表示Teensy 4.1的内存限制。

B. 预计算

在每次ADMM迭代中，求解每个原问题更新的线性系统是最耗时的步骤。在我们的案例中，这相当于求解Riccati方程，而该方程具有一些特性，我们可以利用这些特性显著减少计算量和内存使用。当预测时域足够长时，Riccati递推公式（3）会收敛到无限时域LQR问题的常数解[33]。因此，我们预先计算一个单一的LQR增益矩阵K_{inf}和代价函数Hessian矩阵P_{inf}。然后，我们缓存来自公式（3）的以下矩阵：

$$\begin{aligned} C_1 &= (R + B^\intercal P_{\text{inf}} B)^{-1}, \\ C_2 &= (A - BK_{\text{inf}})^\intercal. \end{aligned} \tag{19}$$

对Riccati方程进行仔细分析后发现，在ADMM迭代过程中，只需更新线性项即可：

$$\begin{aligned} d_k &= C_1(B^\intercal p_{k+1} + r_k), \\ p_k &= q_k + C_2 p_{k+1} - K_{\text{inf}}^\intercal r_k. \end{aligned} \tag{20}$$

因此，我们完全避免了在线矩阵分解，仅计算矩阵与向量的乘积。同时，通过在每个时间步只存储少量向量，显著降低了内存占用。

C. 惩罚缩放

ADMM 对公式 (9) 中惩罚项 ρ 的取值较为敏感。像 OSQP [31] 这样的求解器通过自适应地调整 ρ 来克服这一问题，但这需要进行额外的矩阵分解。为了避免这种情况，我们预先计算并缓存了对应于多个 ρ 值的一组矩阵。在线上，我们根据原始残差和对偶残差的值，在基于OSQP改进的方案中切换这些缓存的矩阵。由此得到的TinyMPC算法总结在算法1中。

Algorithm 1 TinyMPC

function TINY_SOLVE(input)
  while not converged **do**
    //Primal update
    $p_{1:N-1}, d_{1:N-1} \leftarrow$ Backward pass via (20)
    $x_{1:N}, u_{1:N-1} \leftarrow$ Forward pass via (2)
    //Slack update
    $z_{1:N}, w_{1:N-1} \leftarrow$ Project to feasible set (17)
    //Dual update
    $y_{1:N}, g_{1:N-1} \leftarrow$ Gradient ascent (18)
    $q_{1:N}, r_{1:N-1}, p_N \leftarrow$ Update linear cost terms
  return $x_{1:N}, u_{1:N-1}$

IV. 实验

我们通过两组实验来评估TinyMPC：首先，我们在一个典型的微控制器上将我们的求解器与最先进的OSQP[31]求解器进行基准测试，证明了其在计算速度上的提升以及内存占用的减少。随后，我们在资源受限的纳米四旋翼平台Crazyflie 2.1上测试了我们求解器的有效性。结果表明，TinyMPC能够使Crazyflie在满足控制限制和时变状态约束的同时，跟踪具有挑战性的参考轨迹。

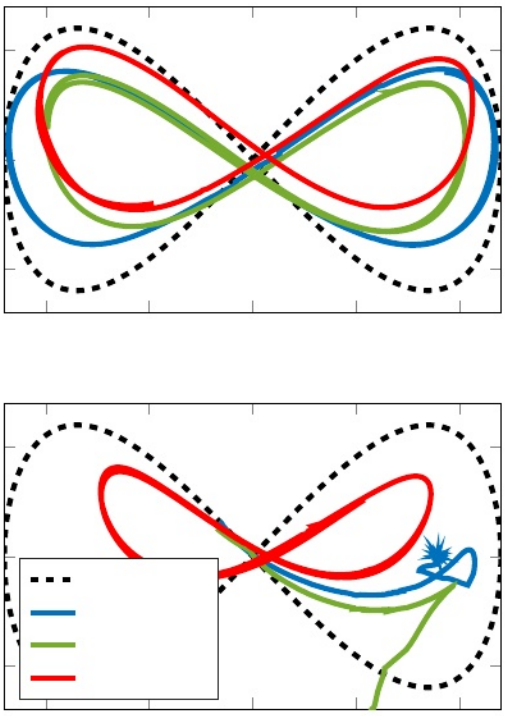


图4. 在低速（上）和高速（下）条件下，TinyMPC与Crazyflie上性能最优的两种控制器进行八字形轨迹跟踪对比。对于较慢的轨迹，三种控制器表现出相似的性能；而对于较快的轨迹，只有TinyMPC能够保持稳定跟踪而不发生碰撞。使用TinyMPC时，最大速度和悬停状态下的姿态偏差分别达到1.5 m/s和20°。

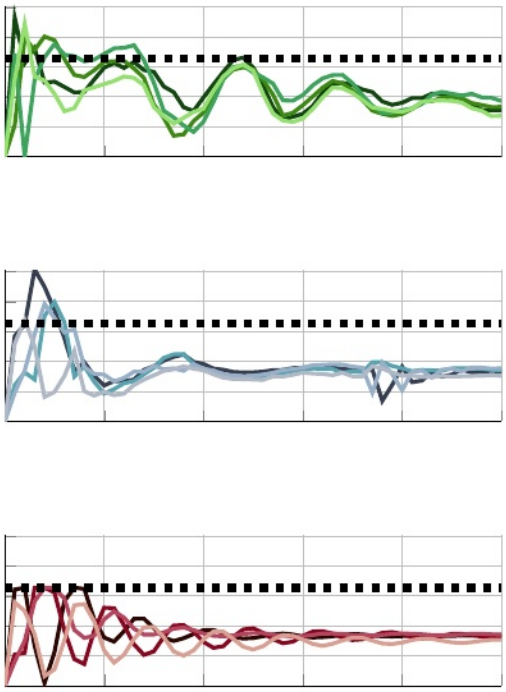


图5. 极端初始姿态实验中的控制轨迹。每个控制器显示了四个预剪裁的PWM电机指令。黑色虚线表示推力限制，范围为0到65535的PWM值。在三个成功的控制器中，只有TinyMPC能够推理控制可行性，并表现出图1（底部）所示的动作。

A. 单片机性能测试

如图3所示，我们首先在随机线性MPC问题上比较TinyMPC和OSQP的性能，同时改变状态和输入维度以及预测时域长度。

1) 方法论：实验在Teensy 4.1 [43] 开发板上进行，该开发板搭载了一款主频为600 MHz的ARM Cortex-M7微控制器，配备7.75 MB闪存和512 kB RAM。TinyMPC使用C++语言结合Eigen矩阵库[44]实现。我们利用OSQP的代码生成功能，为每个问题生成可在微控制器上运行的C语言实现。目标函数容差设为10^{-3}，约束容差设为10^{-4}。两个求解器的最大迭代次数均设为4000次，并均启用了热启动。为减少求解时间，禁用了OSQP的解优化功能。其他参数尽可能设置为等效值。

动力学模型A和B是随机生成的，并经过验证，确保在所有状态维度n、输入维度m和时间范围N的取值下均具有可控性。控制输入在整个时间范围内被限制在固定的边界内。在微控制器测试过程中，添加了噪声以模拟不完美的状态估计。最大的问题实例包含696个决策变量、490个线性等式约束和392个线性不等式约束。

2) 评估：图3展示了两种求解器的平均执行时间，其中TinyMPC相比OSQP实现了最高达8.85倍的速度提升。这一加速效果使TinyMPC能够在处理输入约束的同时实现实时轨迹跟踪。此外，OSQP很快便超过了MCU的内存限制，而TinyMPC则能够扩展到更大的问题规模。例如，在输入维度固定为m = 4、时间范围为N = 10的情况下（图3a），当状态维度n仅为16时，OSQP就已超出Teensy芯片512 kB的内存限制，而TinyMPC在状态维度n = 32时仅使用了约400 kB内存。

B. 硬件实验

我们展示了我们的求解器在资源受限的Crazyflie 2.1四旋翼飞行器上实时执行动态控制任务的有效性。我们进行了三项实验：1）低速和高速下的八字形轨迹跟踪，2）从极端初始姿态恢复，以及3）通过在线更新状态约束实现动态避障。

1) 方法论：Crazyflie 2.1 是一款重量为 27 克的四旋翼飞行器。其主微控制器为运行在 168 MHz 频率下的 ARM Cortex-M4（STM32F405），配备 192 kB 的 SRAM 和 1 MB 的闪存。由于 OSQP 无法在该微控制器的可用内存中运行，因此无法将其作为 MPC 的基准进行使用。相反，我们将其与四种控制器进行比较

随 Crazyflie 固件附带的控制器包括：级联 PID [45]、Mellinger [46]、INDI [47] 和 Brescianini [48]。这些是反应式控制器，通常会将控制输入限制在硬件约束范围内。

所有实验均在Opti-Track动作捕捉环境中进行，以100 Hz的频率将姿态数据发送至Crazyflie。对于八字形轨迹跟踪任务和姿态恢复任务，TinyMPC以500 Hz运行，预测时域长度为N = 15。在避障任务中，我们通过机载无线电将一根棍子末端的位置信息发送给Crazyflie。此外，我们将MPC的运行频率降低至100 Hz，并将N增加至20。

在所有实验中，我们以悬停状态为基准对四旋翼的六自由度动力学进行线性化，并采用文献[49]中的公式用四元数表示其姿态。该问题的状态维度为n = 12，m = 4，分别代表四旋翼的完整状态和PWM电机控制指令。其中规模最大的问题是动态避障场景，该问题在飞行器上以高频率求解，包含316个决策变量、248个线性等式约束和172个线性不等式约束。

2) 评估——八字形轨迹跟踪：我们使用图4所示的八字形轨迹，比较了TinyMPC与其他控制器的跟踪性能。对于较快的轨迹，最大速度和姿态偏差分别达到1.5 m/s和20°。只有TinyMPC能够在满足执行器限制的前提下完整跟踪整个参考轨迹，而Mellinger和Brescianini控制器几乎立即发生碰撞。TinyMPC在每一步最多7次迭代内收敛，并且在由500 Hz控制频率规定的2 ms求解时间内完成计算。

3) 评估——极端初始姿态：图1（底部）展示了Crazyflie在初始姿态存在90°误差时的性能表现。TinyMPC在遵守输入限制的前提下，表现出最佳的恢复能力，最大位置误差仅为23厘米。而PID和Brescianini控制器分别产生了40厘米和65厘米的最大误差，并且违反了输入限制（见图5）。其他控制器INDI和Mellinger则未能稳定四旋翼飞行器，导致其坠毁。

4) 评估——动态避障：我们通过让TinyMPC避开一根移动的棍子（图1上部）来展示其处理时变状态约束的能力。这些实验更具挑战性，因为约束条件在非激活与激活状态之间任意切换，需要更多迭代才能收敛求解。在每一步MPC计算中，障碍物球体均以其更新后的位置重新线性化，从而使无人机能够避开摆动棍子的非预期运动。如图所示，四旋翼能够在空间中自由移动以避开动态障碍，并安全返回悬停位置。作为额外挑战，我们增加了一个约束条件，要求四旋翼必须保持在由x = 0定义的垂直平面内。尽管如此，Crazyflie在成功避开动态障碍的同时，偏离该约束平面的最大距离约为5厘米。

V. 结论与未来工作

我们介绍了TinyMPC，这是一种面向资源受限嵌入式系统的模型预测控制求解器。TinyMPC采用ADMM方法处理状态和输入约束，同时利用MPC问题的结构特性以及LQR的优化思路，相比OSQP等现有先进求解器显著减少了内存占用并加快了在线执行速度。我们在Crazyflie微型四旋翼飞行器上验证了TinyMPC的实际性能，该飞行器在存在输入约束和障碍物约束的情况下执行了高度动态的任务。

未来仍有几个研究方向值得探索。首先，将TinyMPC扩展以支持二阶锥约束应较为直接，这类约束在许多模型预测控制（MPC）应用中用于建模推力和摩擦力非常有用。此外，我们计划通过开发定点版本进一步降低TinyMPC的硬件要求，因为许多小型微控制器缺乏硬件浮点运算支持。最后，为便于部署和推广，我们计划为TinyMPC开发一个基于高级语言（如Julia或Python）的代码生成封装工具，类似于OSQP和CVXGEN。

如需了解更多信息并开始使用我们的开源求解器，我们建议用户访问我们的网站 https://tinympc.org。

VI. 致谢

作者感谢布莱恩·杰克逊的深入讨论，以及马克·贝迪利翁教授为我们提供的额外Crazyflies无人机。

参考文献

[1] P. M. Wensing, M. Posa, Y. Hu, A. Escande, N. Mansard, and A. Del Prete, "Optimization-based control for dynamic legged robots," *arXiv preprint arXiv:2211.11644*, 2022.

[2] J. Di Carlo, "Software and control design for the mit cheetah quadruped robots," Ph.D. dissertation, Massachusetts Institute of Tech-nology, 2020.

[3] Z. Manchester, N. Doshi, R. J. Wood, and S. Kuindersma, "Contact-implicit trajectory optimization using variational integrators," *The International Journal of Robotics Research*, vol. 38, no. 12-13, pp. 1463–1476, 2019.

[4] S. Kuindersma, "Taskable agility: Making useful dynamic behavior easier to create," Princeton Robotics Seminar, 4 2023.

[5] B. Plancher and S. Kuindersma, "A performance analysis of parallel differential dynamic programming on a gpu," in *Algorithmic Foundations of Robotics XIII: Proceedings of the 13th Workshop on the Algorithmic Foundations of Robotics 13*. Springer, 2020, pp. 656–672.

[6] S. M. Neuman, B. Plancher, T. Bourgeat, T. Tambe, S. Devadas, and V. J. Reddi, "Robomorphic computing: A design methodology for domain-specific accelerators parameterized by robot morphology," ser. ASPLOS 2021. New York, NY, USA: Association for Computing Machinery, 2021, p. 674–686. [Online]. Available: https://doi-org.ezp-prod1.hul.harvard.edu/10.1145/3445814.3446746

[7] K. McGuire, C. De Wagter, K. Tuyls, H. Kappen, and G. C. de Croon, "Minimal navigation solution for a swarm of tiny flying robots to explore an unknown environment," *Science Robotics*, vol. 4, no. 35, p. eaaw9710, 2019.

[8] S. D. De Rivaz, B. Goldberg, N. Doshi, K. Jayaram, J. Zhou, and R. J. Wood, "Inverted and vertical climbing of a quadrupedal microrobot using electroadhesion," *Science Robotics*, vol. 3, no. 25, p. eaau3038, 2018.

[9] B. P. Duisterhof, S. Li, J. Burgu´es, V. J. Reddi, and G. C. de Croon, "Sniffy bug: A fully autonomous swarm of gas-seeking nano quadcopters in cluttered environments," in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2021, pp. 9099–9106.

[10] W. G. et al. Crazyflie 2.0 quadrotor as a platform for research and education in robotics and control engineering. [Online]. Available: https://www.bitcraze.io/papers/giernacki draft crazyflie2.0.pdf

[11] Petoi, "Open source, programmable robot dog bittle," Available at https://www. petoi. com/pages/bittle-open-source-bionic-robot-dog(5. 9. 2023).

[12] S. M. Neuman, B. Plancher, B. P. Duisterhof, S. Krishnan, C. Banbury, M. Mazumder, S. Prakash, J. Jabbour, A. Faust, G. C. de Croon, *et al.*, "Tiny robot learning: challenges and directions for machine learning in resource-constrained robots," in *2022 IEEE 4th International Conference on Artificial Intelligence Circuits and Systems (AICAS)*. IEEE, 2022, pp. 296–299.

[13] Z. Zhang, A. A. Suleiman, L. Carlone, V. Sze, and S. Karaman, "Visual-inertial odometry on chip: An algorithm-and-hardware co-design approach," 2017.

[14] V. Adajania, S. Zhou, S. Arun, and A. Schoellig, "Amswarm: An alternating minimization approach for safe motion planning of quadro-tor swarms in cluttered environments," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 1421– 1427.
[15] P. Varshney, G. Nagar, and I. Saha, "Deepcontrol: Energy-efficient control of a quadrotor using a deep neural network," in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2019, pp. 43–50.

[16] N. O. Lambert, D. S. Drew, J. Yaconelli, S. Levine, R. Calandra, and K. S. Pister, "Low-level control of a quadrotor with deep model-based reinforcement learning," *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 4224–4230, 2019.

[17] C. E. Luis, M. Vukosavljev, and A. P. Schoellig, "Online trajectory generation with distributed model predictive control for multi-robot motion planning," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 604–611, 2020.

[18] L. Xi, X. Wang, L. Jiao, S. Lai, Z. Peng, and B. M. Chen, "Gto-mpc-based target chasing using a quadrotor in cluttered environments," *IEEE Transactions on Industrial Electronics*, vol. 69, no. 6, pp. 6026– 6035, 2021.

[19] G. Torrente, E. Kaufmann, P. F¨ohn, and D. Scaramuzza, "Data-driven mpc for quadrotors," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 3769–3776, 2021.

[20] K. Y. Chee, T. Z. Jiahao, and M. A. Hsieh, "Knode-mpc: A knowledge-based data-driven predictive control framework for aerial robots," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 2819–2826, 2022.

[21] N. T. Jafferis, E. F. Helbling, M. Karpelson, and R. J. Wood, "Untethered flight of an insect-sized flapping-wing microscale aerial vehicle," *Nature*, vol. 570, no. 7762, pp. 491–495, 2019.

[22] B. Goldberg, R. Zufferey, N. Doshi, E. F. Helbling, G. Whittredge, M. Kovac, and R. J. Wood, "Power and control autonomy for high-speed locomotion with an insect-scale legged robot," *IEEE Robotics and Automation Letters*, vol. 3, no. 2, pp. 987–993, 2018.

[23] G. Loianno, C. Brunner, G. McGrath, and V. Kumar, "Estimation, control, and planning for aggressive flight with a small quadrotor with a single camera and imu," *IEEE Robotics and Automation Letters*, vol. 2, no. 2, pp. 404–411, 2016.

[24] Unitree, "Unitree go1," Accessed 2023, https://shop.unitree.com.

[25] Bitcraze, "Crazyflie 2.1," 2023. [Online]. Available: https://www.bitcraze.io/products/crazyflie-2-1/

[26] M. Bechtel, Q. Weng, and H. Yun, "Deeppicarmicro: Applying tinyml to autonomous cyber physical systems," in *2022 IEEE 28th International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA)*. IEEE, 2022, pp. 120–127.

[27] L. Meier, P. Tanskanen, L. Heng, G. H. Lee, F. Fraundorfer, and M. Pollefeys, "Pixhawk: A micro aerial vehicle design for autonomous flight using onboard computer vision," *Autonomous Robots*, vol. 33, pp. 21–39, 2012.

[28] Petoi, "Petoi bittle robot dog," Accessed 2023, https://www.petoi.com/products/petoi-bittle-robot-dog.

[29] J. L. Jerez, P. J. Goulart, S. Richter, G. A. Constantinides, E. C. Kerrigan, and M. Morari, "Embedded online optimization for model predictive control at megahertz rates," *IEEE Transactions on Automatic Control*, vol. 59, no. 12, pp. 3238–3251, 2014.

[30] B. O'Donoghue, G. Stathopoulos, and S. Boyd, "A splitting method for optimal control," *Control Systems Technology, IEEE Transactions on*, vol. 21, pp. 2432–2442, 11 2013.

[31] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd, "Osqp: An operator splitting solver for quadratic programs," *Mathematical Programming Computation*, vol. 12, no. 4, pp. 637–672, 2020.

[32] J. Mattingley and S. Boyd, "CVXGEN: A code generator for embedded convex optimization," in *Optimization Engineering*, pp. 1–27.

[33] F. L. Lewis, D. Vrabie, and V. Syrmos, "Optimal Control," 1 2012. [Online]. Available: https://doi.org/10.1002/9781118122631

[34] B. Ac¸ıkmes¸e, J. M. Carson, and L. Blackmore, "Lossless convexification of nonconvex control bound and pointing constraints of the soft landing optimal control problem," *IEEE Transactions on Control Systems Technology*, vol. 21, no. 6, pp. 2104–2113, 2013.

[35] J. Di Carlo, P. M. Wensing, B. Katz, G. Bledt, and S. Kim, "Dynamic locomotion in the mit cheetah 3 through convex model-predictive control," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018, pp. 1–9.

[36] M. Babu, Y. Oza, A. K. Singh, K. M. Krishna, and S. Medasani, "Model predictive control for autonomous driving based on time scaled collision cone," in *2018 European Control Conference (ECC)*, 2018, pp. 641–648.

[37] A. Boccia, L. Gr¨une, and K. Worthmann, "Stability and feasibility of state constrained mpc without stabilizing terminal constraints," *Systems and Control Letters*, vol. 72, pp. 14–21, 2014. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167691114001595
[38] A. Mohammadi, H. Asadi, S. Mohamed, K. Nelson, and S. Nahavandi, "Optimizing model predictive control horizons using genetic algorithm for motion cueing algorithm," *Expert Systems with Applications*, vol. 92, pp. 73–81, 2018. [Online]. Available: https: // www.sciencedirect.com/science/article/pii/S0957417417306000

[39] R. Glowinski and A. Marroco, "Sur l'approximation, par e´l´ements finis d'ordre un, et la r´esolution, par p´enalisation-dualit´e d'une classe de probl`emes de dirichlet non lin´eaires," *Revue franc¸aise d'automatique, informatique, recherche op´erationnelle. Analyse num´erique*, vol. 9, no. R2, pp. 41–76, 1975.

[40] D. Gabay and B. Mercier, "A dual algorithm for the solution of nonlinear variational problems via finite element approximation," *Computers & mathematics with applications*, vol. 2, no. 1, pp. 17– 40, 1976.

[41] S. Boyd, N. Parikh, E. Chu, B. Peleato, J. Eckstein, *et al.*, "Dis-tributed optimization and statistical learning via the alternating di-rection method of multipliers," *Foundations and Trends® in Machine learning*, vol. 3, no. 1, pp. 1–122, 2011.

[42] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein, 2011.

[43] "Teensy® 4.1." [Online]. Available: https://www.pjrc.com/store/teensy41.html

[44] G. Guennebaud, B. Jacob, *et al.*, "Eigen v3," http://eigen.tuxfamily.org,2010.

[45] "Controllers in the Crazyflie — Bitcraze." [Online]. Available: https://www. bitcraze. io/documentation/repository/crazyflie-firmware/master/functional-areas/sensor-to-control/controllers/

[46] D. Mellinger and V. Kumar, "Minimum snap trajectory generation and control for quadrotors," in *2011 IEEE International Conference on Robotics and Automation*, 2011, pp. 2520–2525.

[47] E. J. J. Smeur, Q. Chu, and G. C. H. E. de Croon, "Adaptive incremental nonlinear dynamic inversion for attitude control of micro air vehicles," *Journal of Guidance, Control, and Dynamics*, vol. 39, no. 3, pp. 450–461, 2016.

[48] D. Brescianini, M. Hehn, and R. D'Andrea, "Nonlinear quadrocopter attitude control," 2013.

[49] B. E. Jackson, K. Tracy, and Z. Manchester, "Planning with attitude," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 5658–5664, 2021.